

C语言文件简易操作教程

文件

文件是存储在计算机上的数据的集合，可以被我们的电脑解析为文本文件或可执行文件等不同类型。它们可以被我们反复访问，一般也能够长久存在于你的电脑中（只要不被删除）。

我们执行的C语言程序（windows上编译生成的 `hello.exe` 或Linux上编译生成的 `a.out`）其实也是文件，一般被称为可执行文件。当我们执行它们时，操作系统（Windows、Linux等）将会为程序提供一份临时的空间用于其运行，用来存储其运行时的各种数据。既然是临时空间，那么在我们的程序执行期间储存的各种数据都会在程序结束后“消失”，至少我们不能轻易地再得到它们。

如果这些数据我们不需要长久保存也罢，但有许多数据我们希望它能够长久保存，最好让它永远存在下去，这该如何呢？

聪明的你想到一种方法——输出并重定向，我们可以让程序用一定的规则将我们需要的数据输出到终端。在终端里，我们可以使用重定向符（`>`、`>>`）来让程序的输出导入到特定文件中。但这样的局限性非常大，在没有特殊处理的情况下，该程序的所有输出都会被重定向到单独的一个文件中，也不能随心所欲地控制我们要导入哪些数据到文件中。

更加局限的是，我们通过这种方法很难将这些数据重新读回到程序当中，体现在游戏中就是我们的就算导出了存档但又无法读取存档，而且我们还只能在退出前存入一个存档。

那么我们的游戏或者其他各种程序是如何利用文件存取数据的呢？很明显，我们需要能够在代码中操作文件的功能——C语言文件操作。这些操作不仅高效，还十分符合我们日常对文件的使用习惯。

打开文件

同日常使用一样，我们首先考虑如何打开文件，但现在我们还无从下手。

我们已经学习过了如此多的变量类型，像是 `int` / `char` / `struct xxx`，那么有没有什么变量类型能够帮助我们来操作文件呢？

C语言标准库 `stdio.h` 里面预定义了一个 **结构体类型** —— `FILE`，看名字就知道，这正是用于文件操作的一种结构体，C语言程序就是通过它来管理文件的。

那么我们是不是可以通过创建一个 `FILE` 结构体变量来做到操作文件的功能呢？其实也不能，C语言标准规范了程序员的行为，程序员不能通过直接访问 `FILE` 结构体来进行文件操作。至于为何如此？我们将在未来继续深入学习。

为了能够让程序员使用 `FILE` 结构体，C语言标准选择使用它信任的库函数来解决这个问题。同时为了不让程序员能够直接访问 `FILE` 类型变量，所以关于 `FILE` 的任何操作都只能通过其指针（`FILE*`）来进行。而像这样指向结构体的指针就是结构体指针，这里的结构体就是 `FILE`，这样的指针的类型就是 `FILE*`。

就像你能够通过一个 `int*` 类型的指针来修改/访问 `int` 数组中的元素一样，`FILE*` 这样的结构体指针也能通过某些方式来修改其指向的结构体中的成员数据，并以此来影响我们的外部文件，实现修改/读取外部文件的功能。

回到最开始的问题，我们如何打开文件？C语言中打开文件通常使用 `fopen` 函数（file open），我们给出 `fopen` 函数声明如下：

代码块

```
1 FILE *fopen(const char *filename, const char *modes);
```

不过这样使用结构体时并不需要使用 `struct` 关键字，因为 `FILE` 其实是对一个结构体类型 `typedef` 的结果。如果你还不知道 `typedef`，你可以自行[STFW了解](#)其作用和使用。

对于这个声明，其中的第一个参数需要放入一个表示文件路径的字符串，第二个参数则是以字符串形式传入的 `模式`。同时，该函数的返回值是一个结构体指针，指向一个 `FILE` 类型的结构体。

它就像是一个“文件遥控器”，它不是文件本身，而是你用来打开、读取、写入、关闭文件的一个“工具”，就像你用遥控器控制电视一样，你在C程序中用 `FILE*` 变量来控制一个文件。

第一步

1. 因为 `FILE *` 是 `fopen` 的返回值，所以我们需要用 `FILE *` 创建一个变量。此时应该将变量初始化为 `NULL`，代表其值是空的。

代码块

```
1 FILE *fp = NULL;
```

2. 然后让变量赋值为 `fopen` 的返回值。我们假设文件名为 `"file.txt"`，其中的 `"w"` 表示打开模式为“覆盖打开”。

代码块

```
1 FILE *fp = NULL;
2 fp = fopen("file.txt", "w");
```

这样我们就在C语言程序打开了一个名为 `file.txt` 的文件。文件名好理解，但模式（modes）为什么是 `"w"` 呢？这是C语言定义的文件打开模式。`fopen` 不仅仅用于打开文件，也会根据不同的打开模式规定不同的行为，这发生在文件打开之前。比如 `"w"` 模式，我称其为“覆盖打开”，在理解它们后你也可以为这些模式起一个易懂的名字。

fopen常见模式

模式	描述
"w"	创建一个 仅 用于写入的空文件。如果文件名称与已存在的文件相同，则会删除已有文件的内容，文件的空文件。
"r"	打开一个 仅 用于读取的文件，且该文件必须存在。
"a"	追加到一个文件。写操作向文件末尾追加数据。如果文件不存在，则创建文件。同样 不能用于读取 。
"r+"	打开一个 用于更新内容 的文件，可读取也可写入。该文件必须存在。
"w+"	覆盖创建一个用于读写的空文件。
"a+"	打开一个用于读取和追加的文件。

而下面更简单的表格可用于你未来进行查阅：

模式	描述	文件是否需要存在？	是否会清空原文件？
"r"	只读模式打开文本文件,仅用于读取	是	否
"w"	只写模式打开文本文件,仅用于写入	否	是（若存在则清空）
"a"	追加写入模式打开文本文件,在文件末尾追加写入	否	否（写入时追加）
"r+"	读写模式打开文本文件，可读取也可写入	是	否
"w+"	读写模式，若文件存在则清空	否	是（若存在则清空）
"a+"	读写模式，从文件末尾追加写入	否	否（写入时追加）

文件操作

完成对文件的打开后，我们就该考虑如何读取文件数据或者向其中写入数据了。

和我们用于终端中交互的 `scanf` 和 `printf` 类似，`fscanf` 和 `fprintf` 也有相似的功能。

`fprintf` 函数和 `fscanf` 函数的声明如下，在下方对应放置了 `printf` 函数和 `scanf` 函数的声明：

代码块

```
1  int fprintf (FILE *stream, const char *format, ...);
2  int  printf (          const char *format, ...);
```

代码块

```
1 int fscanf (FILE *stream, const char *format, ...);
2 int scanf (const char *format, ...);
```

原来在函数前面加了个 `'f'` 修饰，仅仅是在调用时将 `FILE` 文件 插个队到前面作为第一个参数。那么这会有什么不一样呢？

聪明的你一定想到了，这应该是把原本要从终端 输入/输出 的东西转而变到在这个文件里工作。事实也的确是如此。更加规范地，我们把 输入/输出 叫做 `I/O`，也就是 `input/output` 的简写。这两个函数的作用就是把 `I/O` 暂时重定向到这个文件中。

第二步

刚刚完成了第一步的打开文件，现在就是如何操作文件了。我们先尝试把以下四行紫色文本数据写入 `file.txt` 中。

代码块

```
1 1 1 2
2 2 1 3
3 4 5 9
4 3 4 9
```

我们得到了如下代码：

代码块

```
1 FILE *fp = NULL;
2 fp = fopen("file.txt", "w");
3 int a[4] = {1,2,4,3};
4 int b[4] = {1,1,5,4};
5 int result[4] = {2,3,9,9};
6 for(int i = 0; i < 4; i++) {
7     fprintf(fp, "%d %d %d\n", a[i], b[i], result[i]);
8 }
```

这样我们就得到了能够“覆盖创建”一个内容为上述文档内容的C语言程序。但刚刚也提到过，这个程序是没有读文件权限的，所以我们用需要另一个程序去读取 `file.txt`。

读文件程序

刚刚已经实现了能够向 `file.txt` 中覆盖创建再写入数据的程序，现在你需要仿照 `scanf` 函数，使用 `fscanf` 函数写一个从 `file.txt` 中读取数据的程序。

不仅如此，你还需要新增一个判断函数，在读取到的一行数据**不符合加法规则**时，输出**计算错误** 或者 `error calculation` 的信息并**附上行号**，像这样：

```
error calculation : line 4
```

如果你仍然无法理解如何使用 `fscanf()`，请自行 **STFW** 找到 `fscanf()` 的教程。

现在我们只需要检查，读取的数据不需要特别地储存起来，所以你还不需要使用数组。



（支线）为什么不用更改 `fp` 指针就能够向后继续读写？

要是你认真学习了上周的指针讲义，你一定会发现我们传入的是 `fp` 这个指针变量作为形参，我们显然不能修改这个指针本身，那这是怎么做到的呢？

上文我们提到 `FILE` 其实是一个结构体，它包含了管理文件所需的各种信息，其中最重要的一个成员就是 **文件位置指示器**（`file position indicator`，以下简称 `file_pi`）。而我们传入函数的是指向这个结构体的指针。fxxx类函数（如 `fscanf()`）通过这个指针找到其指向的结构体变量，再根据 `file_pi` 所记录的位置找到读写位置，然后根据你的代码行为进行读写操作并自动更新 `file_pi`，从而实现了我们继续读写的行为。

但这并不意味着我们可以通过访问该结构体的 `file_pi` 来自己更改文件读写位置。相反C语言禁止我们直接访问 `FILE` 结构体。

更多知识就需要你**STFW**和**RTFM**了，这已经远远超出了讲义要求你掌握的知识范围。

文件关闭

第三步

我们在使用电脑时都知道，用不上的窗口就要关掉。C语言的文件操作也是一样的，要是不关掉文件的话还会导致文件不安全和内存泄漏，从而产生我们不期望的行为（比如文件内容被错误地更改）。所以我们需要一个能够关闭文件的函数——`fclose`。

以下是 `fclose` 函数的声明：

代码块

```
1  int fclose (FILE *__stream);
```

显然，我们只需要把一开始获得的fp变量作为参数传入就可以关闭文件了。所以刚刚那个写程序的完整代码应该是这样的：

代码块

```
1  #include <stdio.h>
2
3  int main(){
4      FILE *fp = NULL;
5      fp = fopen("file.txt", "w");
6      int a[4]    = {1,2,4,3};
7      int b[4]    = {1,1,5,4};
8      int result[4] = {2,3,9,9};
9      for(int i = 0; i < 4; i++) {
10         fprintf(fp, "%d    %d    %d\n", a[i], b[i], result[i]);
11     }
12     fclose(fp);
13     fp = NULL;
14     return 0;
15 }
```

明明我们已经关闭了文件，为什么还要将一开始获得的fp变量指向NULL呢，这其实是一种代码规范行为，具体内容将会在未来继续深入学习。

修正

为你刚刚写的读文件程序加上fclose吧。

完成后**将你的源代码重命名为** file 并放入 姓名-专业班级-Great-4 文件夹，等待作业提交。

文件操作（知识拓展）

刚刚你已经通过读文件程序找到了哪一行有错误，但是我们难道要看着错误留存下去吗？尽管你可能还在抱怨是哪个笨蛋写错了加法，但是自动纠错也应该是程序要做的。假如我们要批改纠错的加法程序不仅仅是这四条，而是上百上千条呢？

然后你仔细思考并发现了一个问题，我们的程序好像只能一直往前读，但是不能有任何回头。

或许你会想——我用fopen重新打开一遍再编辑不就好了。但是有时候为了安全考虑，文件打开了就不再允许再次打开，就像是被上了锁，上面写着“使用中”。要是在fclose后重新打开呢？那你就会发现另一个问题——我要怎么存储“问题”呢？要是问题数量已经超出程序能处理的极限怎么办？

所以我们更需要能让程序往回走的函数——fgetpos函数和fsetpos函数。

man!

阅读 `man fgetpos`，找到 `fgetpos` 和 `fsetpos` 的函数声明，然后继续RTFM或通过STFW找到其使用方法。

更改你的读程序为读写程序，在检查到错误时用上述两个函数回到这一行开头并用正确的结果覆盖这一行。不过你可能还需要学习如何储存这一行的信息。

覆盖而不是删除

如果你真的去实现了上面要求的程序，那么你会发现你能做的只有覆盖文件，并不能做到像平时使用文本编辑器一样让这个位置“消失”。无奈的你只好选择用空格来补上这些个洞。然而如何真正删除这部分内容又超出了我们讲义需要你掌握的知识部分，如果你想了解这些知识，可以通过STFW来解决。

? 为什么要用fgetpos和fsetpos

在RTFM后，你一定发现了其他可以用来“回头”的函数。但我们偏偏只选择了其中看似最难用的两个。这是为什么呢？

其实是因为这两个函数涉及到的结构体方面的知识要更多些。比如我们使用的fpos_t类型其实就是一个结构体类型。我们希望你能去探索这个结构体中的成员。

? 其实这次讲义还教了你一个好的命名修饰规则，你有没有发现呢？好的命名规则会有助于你自己和读代码的人记忆函数的使用。